

FOODRESCUE — WHITEPAPER

Custody *without a custodian*

for agricultural surplus

V1.0

MVP ON SOLANA DEVNET

SEPTEMBER 2026

IN THIS DOCUMENT

Contents

1	Waste is a trust problem	
2	Thesis	
3	Protocol design	
4	The five actors	
5	Lifecycle of a trade	
6	Donation and Proof of Rescue	
7	Protocol economics	
8	Trust model	
9	Out of scope for the MVP	
10	Roadmap	

SECTION 1

Waste is a trust problem

The FAO estimates that roughly a third of the world's food production is lost between harvest and plate. In Brazil, a significant share of that loss happens **before the farm gate**: the lot exists, it is good, it has a potential buyer — and it rots anyway.

The bottleneck is rarely physical. It is contractual:

- **The producer** has a lot with a short window and no channel to move it beyond the usual buyers.
- **The buyer** will not advance money to a seller they do not know, for goods they have not seen.
- **The NGO** would accept the donation, but cannot guarantee anyone that the freight will be paid.
- **The carrier** does not roll without certainty of payment.

Four parties who all want the same deal, and none can afford to trust first. The classic intermediary solves this by charging heavily and concentrating the risk — becoming itself the single point of failure and of margin capture.

SECTION 2

Thesis

If nobody needs to trust anybody, the deal happens.

FoodRescue replaces interpersonal trust with a programmable guarantee. The value of the deal sits locked in an **on-chain vault with no custodian**: an account whose authority is an address derived deterministically from the deal itself (a PDA), with no corresponding private key. Neither the platform, nor the buyer, nor the producer can move that balance outside the rules written into the program.

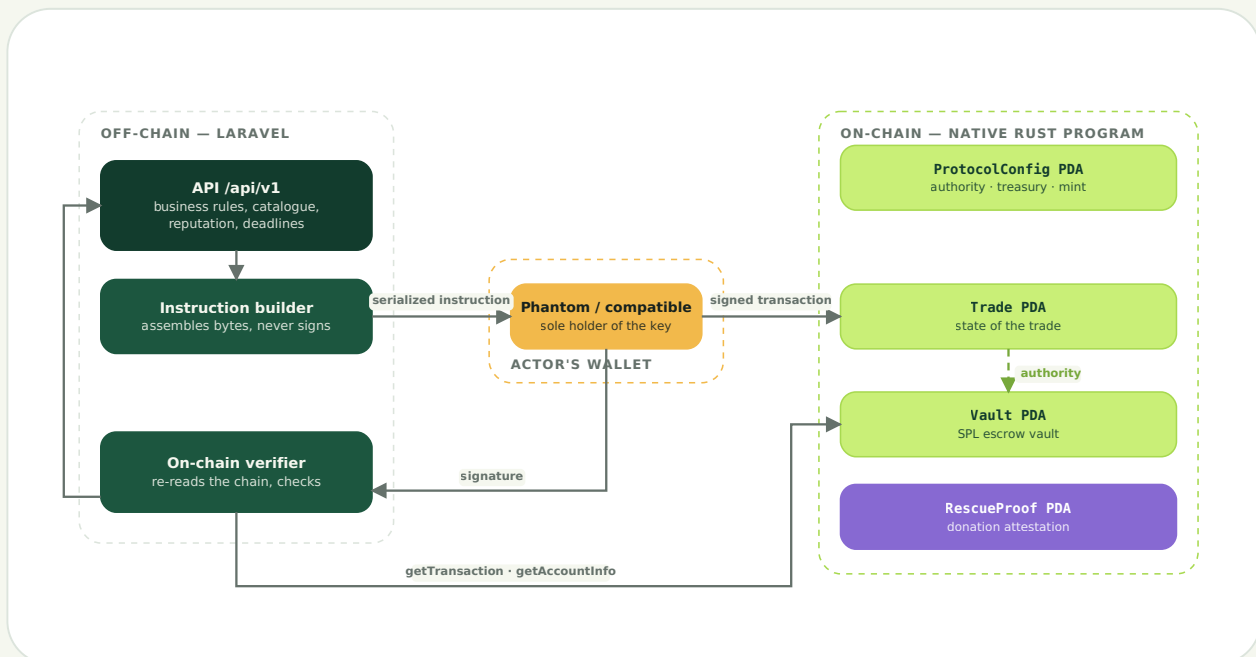
Three practical consequences:

1. **The buyer deposits without fear** — if delivery does not happen, cancellation returns everything.
2. **The producer ships without fear** — the money is already locked before pickup.
3. **The carrier rolls without fear** — the freight leaves the same vault, in the same settlement transaction.

And one structural consequence: **the platform stops being a risk**. It cannot run off with the money because it never held it.

SECTION 3

Protocol design



The protocol has two halves with deliberately separated responsibilities.

Off-chain (Laravel) handles what is expensive and mutable on a chain: product catalogue and grading, marketplace search, offer negotiation, the freight quote market, reputation, operational deadlines, actor profiles. None of that needs global consensus.

On-chain (native Rust program, no Anchor) handles only what **has** to be incontestable: who is entitled to how much, under what condition, and the public record that the rescue happened. Four account types and ten instructions — deliberately small, because every byte of on-chain surface is attack surface.

The seam is the **prepare → sign → verify** pattern:

1. The backend assembles the exact instruction and returns it to the front end, together with the expected addresses and amounts.
2. The actor's wallet signs. The server never sees a private key at any point.
3. The backend receives the signature and **re-reads the chain**: it checks that the transaction exists, was confirmed, contains the right instruction on the right PDA, was signed by whoever it should have been, and that the resulting state matches field by field what it had promised — including the amounts actually transferred.

Only then does the database advance. The byte-by-byte detail of that verification is in the [Yellowpaper](#).

SECTION 4

The five actors

ACTOR	BRINGS	RECEIVES
 Producer	The surplus, signed with their own wallet at publication	Product value minus 2%
 Buyer	The capital, locked in escrow before pickup	The lot, or the money back
 Carrier	A quote in an open market and the execution of the freight	100% of the freight, no deduction
 NGO	The ability to allocate socially, and the cost of the freight	The lot + public proof of the rescue
 Admin	Reference catalogue, deadlines, <code>ProtocolConfig</code>	—

The role defines permissions (RBAC via `spatie/laravel-permission`), but it **never replaces the ownership check**: being a carrier grants no access to someone else's trade; being a producer grants no access to another producer's lot.

Each wallet is bound to an account by an **Ed25519 challenge** carrying a nonce, a purpose and an expiry. Publishing a surplus requires signing again — the lot in the catalogue carries the cryptographic commitment of whoever published it.

SECTION 5

Lifecycle of a trade



Three time windows govern the flow, all configurable by the admin and swept by a scheduler every minute:

- **Freight quotes** (default 240 min) — once expired, the trade falls back to buyer-arranged pickup instead of dying.
- **Payment** (default 15 min) — expired without funding, the lot returns to the catalogue. Expiry is **reconciled against the chain** before it takes effect: if on-chain state exists, the backend does not expire it unilaterally.

- **Lot validity** — after it, the surplus leaves the storefront.

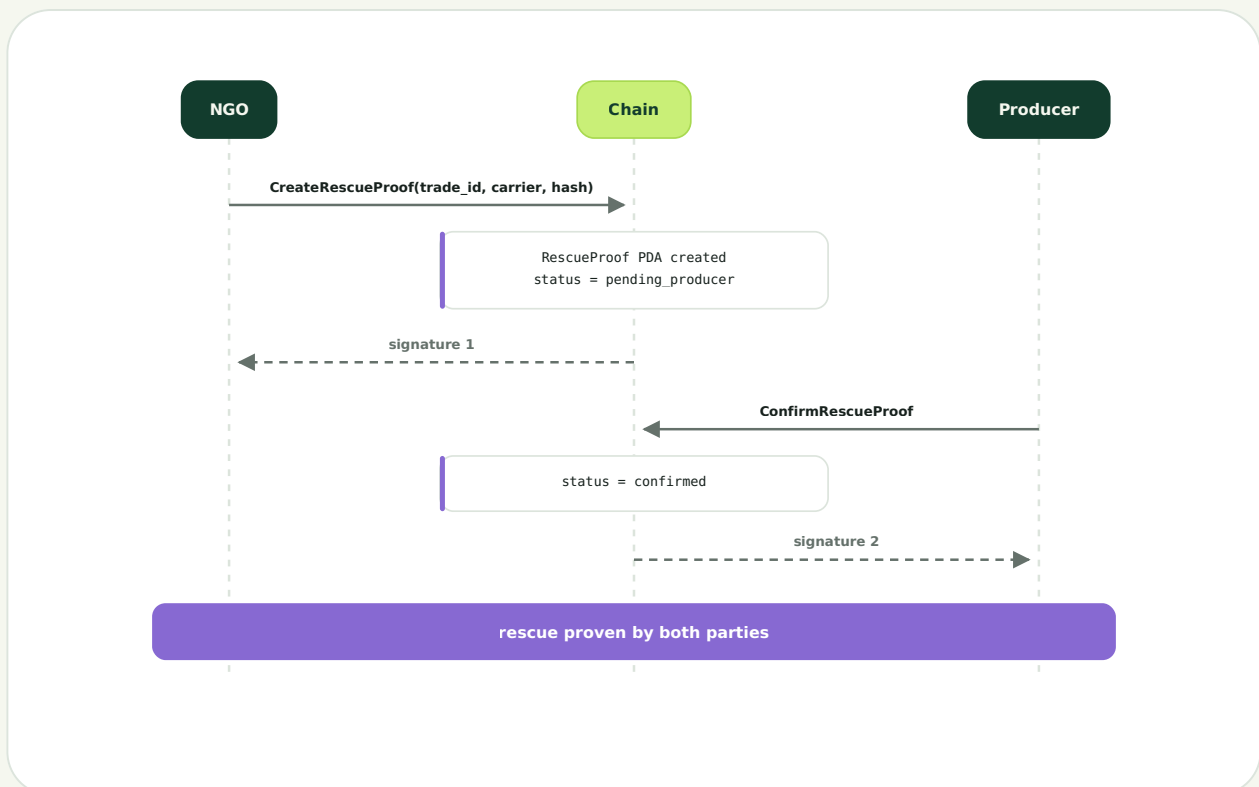
The point of no return is `ready_for_pickup`. Before it, cancellation returns everything to the buyer — but with the escrow already funded, it **requires both parties to sign**. Nobody unwinds alone a deal the other side has already committed to.

SECTION 6

Donation and Proof of Rescue

A donation runs on the same rail, with a product value of zero. If there is freight, the NGO funds the freight alone; settlement pays the carrier and nothing else.

What changes is the ending. A donation with freight does not close in `completed`: it moves to `proof_pending` and only concludes once a **bilateral attestation** exists on-chain.



Why two separate transactions instead of one carrying two signatures? Because the two parties sign at different moments, on different machines — and a shared blockhash expires before that happens. The protocol accepts operational reality instead of fighting it.

The attestation PDA is derived from `(trade_id, NGO wallet, producer wallet)`. Since `trade_id` is predictable, including both wallets in the seeds stops a third party from squatting the address: without the NGO's signature nobody creates its proof, and without the producer's the attestation never leaves "pending".

What gets recorded is not the sensitive data, it is the **SHA-256 hash** of a canonical document holding the lot, product, quantity, unit, parties and freight amount. Auditable by whoever holds the document, opaque to whoever does not.

SECTION 7

Protocol economics

FLOW	DESTINATION
Product — fee	Producer
200 bps (2%) fee on the product	Protocol treasury
Full freight	Carrier
Vault surplus	Returned to the buyer

The fee applies **to the product only**, never to the freight — charging a percentage on logistics would penalise precisely the long routes that move food out of where it is in surplus. And since a donation carries a product value of zero, **social rescue pays no fee at all**. The rule is enforced by the program, not by the platform: an instruction whose fee diverges from the canonical calculation is rejected with `InvalidFee`.

The vault distributes everything in one atomic transaction. Either everyone is paid, or nobody is and the state does not advance — there is no scenario in which the producer is paid and the carrier is left waiting.

The MVP's `FRUSD` is a devnet SPL token created for testing. **It does not represent real fiat currency.** The design, however, is agnostic: any SPL mint registered in `ProtocolConfig` works, including a real stablecoin on an eventual mainnet.

SECTION 8

Trust model

Who has to trust whom, and what happens if that trust fails:

TRUST	SCOPE	IF IT FAILS
None between buyer and producer	The entire financial flow	Nothing — the vault obeys neither of them
The platform, to coordinate	Catalogue, negotiation, deadlines	Existing on-chain trades remain intact
The <code>ProtocolConfig</code> authority	Sets treasury and mint	Misdirected fee; the escrow itself is unaffected
Human oracle: "it was delivered"	Delivery confirmation	See below

The honest point of the design: **physical delivery has no cryptographic proof**. Whoever confirms that the goods arrived is the recipient, signing with their own wallet. The protocol guarantees that the money only moves once that confirmation exists and that it came from the right party — not that the truck actually arrived. This is not a flaw to hide: it is the intrinsic limit of any escrow over a physical good. What can be done, and is done, is to make every step **attributable and public**, and to rest the remainder on reputation accumulated per completed trade.

SECTION 9

Out of scope for the MVP

Stated plainly, because a whitepaper that only promises is no help in deciding anything:

- **Dispute arbitration.** There is no arbiter for "it arrived spoiled". Today the path is bilateral cancellation before pickup.
- **Partial delivery.** The lot is indivisible: the escrow settles whole or is returned whole.
- **Mainnet and a real stablecoin.** `FRUSD` is devnet.
- **Immutable program.** The program's upgrade authority still exists; renouncing it is a mainnet decision.
- **Identity verification.** Documents and addresses are declared, not verified against an external source.

SECTION 10

Roadmap

PHASE	DELIVERY
Now — devnet MVP	Full commercial and donation cycle on-chain, bilateral Proof of Rescue, reputation, impact dashboards
Next — hardening	Isolation tests between actors across the whole money path, strong password policy, CSP, unified authorisation in policies
Then — real pilot	A real stablecoin, renouncing the upgrade authority, external audit of the program
Beyond — scale	Arbitration with an elected arbiter, partial delivery, impact export for auditable ESG reporting

*Less waste. More value and
verifiable impact.*

[Yellowpaper — technical specification →](#)